

Rapport de projet

SmartStudy AI

Une application d'étude intelligente qui transforme des documents en révision active, suivis de progression et outils d'apprentissage.

Angle du projet

Productivité, mémorisation active et suivi pédagogique

Ce que le professeur verra immédiatement

Une architecture claire, des modules bien séparés, un vrai parcours utilisateur et une logique métier cohérente.

1. Vue d'ensemble

SmartStudy AI est une plateforme d'étude intelligente pensée pour convertir un simple dépôt de cours en expérience d'apprentissage guidée. Le projet associe extraction de contenu, génération d'outils de révision et suivi de progression dans une architecture légère, locale et modulaire.

Message à retenir

L'application ne se contente pas de stocker des documents: elle les transforme en support actif de révision, avec des flashcards, des quiz, un chat contextuel et des indicateurs de progression.

2. Architecture globale

Le dépôt est organisé en monorepo avec un backend principal en C et un frontend moderne en Next.js. Le backend expose les API, gère la persistance et orchestre les traitements. Le frontend fournit les écrans d'authentification, le tableau de bord et les vues métier.

Module	Rôle
core/	Initialisation de l'application, configuration, logging et cycle de vie.
db/	Accès SQLite, modèles de données, requêtes CRUD et migrations.
api/	Clients HTTP vers les services externes et compatibilité REST.
auth/	JWT et OAuth GitHub / Google.
chat/	Moteur de conversation contextuel.
pdf/	Extraction et analyse de documents PDF.
review/	Flashcards, quiz, répétition espacée.
progress/	Suivi de la progression et des statistiques.
mindmap/	Génération de cartes mentales.
frontend/	Interface Next.js pour les écrans utilisateur.

3. Point d'entrée et cycle de vie

Le point d'entrée du backend est `src/main.c`. Il parse les arguments CLI, charge le fichier `.env`, initialise l'application et lance le serveur HTTP.

Le struct `SmartStudyApp`, défini dans `include/core/app.h`, sert de contexte global pour conserver le handle de base de données, le chemin de configuration et l'état d'initialisation.

La fonction `app_init()` de `src/core/app.c` charge la configuration, initialise le logger, ouvre SQLite, applique le schéma SQL et démarre le sous-système HTTP.

La fonction `app_shutdown()` ferme proprement la base, nettoie HTTP et libère la configuration.

Point fort technique

Le backend garde une séparation nette entre initialisation, persistance et serveur HTTP. Cela facilite les tests, le débogage et l'évolution du projet.

4. Gestion des données

La couche `include/db/db.h` définit les structures `DBUser`, `DBDocument`, `DBFlashcard` et `DBChatMessage`.

La couche `src/db/db_sqlite.c` fournit les fonctions CRUD, avec requêtes préparées, gestion des résultats et libération mémoire dédiée.

Les migrations sont chargées depuis `data/schema.sql` afin d'initialiser les tables et les contraintes.

SQLite est choisi pour sa simplicité de déploiement et sa compatibilité avec une application locale ou embarquée.

5. Authentification

Le backend expose `/api/auth/register`, `/api/auth/login` et `/api/auth/me`.

L'inscription vérifie l'email, le nom d'utilisateur et la longueur du mot de passe avant de créer le compte.

Les comptes stockent leurs préférences dans `settings_json`, ce qui permet de conserver le nom d'utilisateur et le provider d'authentification.

Le frontend sait basculer entre backend local, Supabase et mode démo selon la disponibilité de l'API.

Lecture par le professeur

Ce bloc montre que l'authentification est pensée comme un vrai système hybride: serveur local en priorité, Supabase en secours, puis mode démo si tout est indisponible.

6. Modules fonctionnels

PDF: extraction du texte à partir de documents importés, puis préparation pour l'analyse.

Chat: moteur conversationnel lié au contexte du document et de la session utilisateur.

Review: génération de flashcards, quiz et répétition espacée.

Progress: calcul d'indicateurs comme le niveau, l'XP et la régularité de travail.

Mindmap: organisation visuelle des idées pour aider à la mémorisation.

L'intérêt de ces modules est de couvrir tout le cycle d'étude: ingestion, compréhension, entraînement et suivi dans le temps.

7. Frontend

Le frontend se trouve dans `frontend/` et utilise Next.js avec des pages dédiées au login, à l'inscription, au tableau de bord, aux documents, au chat, aux flashcards, au quiz et aux paramètres.

Les pages d'authentification gèrent les formulaires, les erreurs réseau, le fallback Supabase et le mode démo.

La page d'accueil met en avant la promesse produit: réviser plus intelligemment, mémoriser plus rapidement.

Le design utilise une esthétique sombre avec halos, gradients et cartes vitrées pour renforcer l'identité du produit.

Pourquoi ça attire l'attention

L'interface a une direction visuelle marquée: contrastes forts, couleurs froides, fonds atmosphériques et cartes lumineuses. Cela donne une impression de produit abouti plutôt qu'un simple prototype.

8. Commandes et exécution

make all: compilation complète du backend C.

make test: exécution de la suite de tests unitaires.

make run: compilation puis lancement de l'application.

npm run dev:web: démarrage du frontend Next.js.

./build/bin/smartstudy --register --email ... --username ... --password ...: création d'un compte depuis le terminal.

Astuce de présentation

Pendant l'oral, tu peux montrer que le projet est exploitable en local, qu'il possède des tests et qu'il couvre toute la chaîne d'usage du document à la révision.

9. Flux de fonctionnement simplifié

- 1) L'utilisateur ouvre le frontend et se connecte ou s'inscrit.
- 2) Il importe un document PDF ou démarre un espace de révision.
- 3) Le backend traite le document, conserve l'état en SQLite et expose les données utiles au frontend.
- 4) L'application génère des flashcards, des quiz, un suivi de progression et des vues de synthèse.
- 5) L'utilisateur revient dans le temps grâce aux révisions et à la progression historisée.

10. Points forts techniques

Architecture modulaire claire entre domaine, persistance, API et interface.

Base SQLite locale simple à déployer.

Fallback d'authentification robuste entre backend local, Supabase et démo.

Jeu de tests existant pour les modules critiques.

Frontend orienté produit avec une identité visuelle forte.

Conclusion pour l'examineur

SmartStudy AI combine une vraie logique de produit, une architecture technique lisible et une expérience utilisateur convaincante. C'est ce mélange qui lui donne une bonne valeur de démonstration.

Conclusion

SmartStudy AI est donc une plateforme d'étude complète: le backend C sert de moteur de traitement et de persistance, le frontend assure l'expérience utilisateur, et les modules métier transforment des documents en outils concrets de révision. Le projet met l'accent sur la productivité, la mémorisation active et le suivi de l'apprentissage.